DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics: Games Engineering

Hardware-accelerated Raytracing in Dynamic Environments using Vulkan

Christian Mulser

DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics: Games Engineering

Hardware-accelerated Raytracing in Dynamic Environments using Vulkan

Hardwarebeschleunigtes Raytracing in dynamischen Umgebungen mittles Vulkan

Author:	Christian Mulser
Supervisor:	Prof. Dr. Rüdiger Westermann
Advisor:	Prof. Dr. Rüdiger Westermann
Submission Date:	October 2, 2025

I confirm that this bachelor's thesis in informatics: games engineering is my own work and I have documented all sources and material used.

Munich, October 2, 2025 Christian Mulser

Abstract

In recent years, real-time, hardware-accelerated ray tracing has made significant progress. Since the release of Nvidia's GeForce RTX series, real-time ray tracing has become increasingly popular in interactive applications. This thesis explores how to implement a hardware-accelerated ray tracer using the Vulkan API. We evaluate different pipelines and strategies for updating acceleration structures in dynamic scenes, analyze their performance, and discuss their advantages and drawbacks. Although the tested approaches are intentionally simple, they provide a solid foundation for understanding how ray tracing can be applied in dynamic, real-time environments, like video games.

Contents

1 Introduction

For a long time, **rasterization** has been the standard for rendering 3D scenes in real-time (e.g., in video games). The basic steps of rasterization are as follows:

1. Every vertex of a 3D object gets transformed in the vertex shader.
2. The transformed triangles are projected onto a 2D pixel grid and a fragment is generated for every pixel that the triangle covers.
3. The fragment shader runs for every generated fragment. It uses the fragment's data and some global data (e.g., light sources) to determine the final color of the pixel.

Rasterization is very fast and efficient, but it also has its downsides. When determining the color of a pixel, we only have access to the data in the current fragment and some global data. The fragment is not aware of other objects in the scene, which makes effects like shadows and reflections very difficult, requiring a workaround like shadow-mapping and screen-space-reflections. In recent years, a new way to do real-time rendering slowly emerged, which, of course, is **ray tracing**.

Ray tracing works by casting rays through the screen's pixels into the 3D scene to determine the surface to be displayed. When a ray hits a surface, secondary rays, such as shadow rays, reflection rays, or refraction rays, can be cast. That is much closer to real life, where light rays bounce between objects until they reach our eyes or a camera lens, and because of that, it is much easier to generate realistic-looking images. However, it was never suitable for real-time applications in the past due to the enormous amount of processing necessary.

Even though ray tracing is relatively new in the realm of real-time graphics, the concept of ray tracing is almost as old as rasterization. In 1968, Arthur Appel explores the idea of casting a ray through a pixel to determine which surface is visible, and also the idea of casting a ray from a surface point to the light source to check if the light reaches the surface unobstructed [App68]. Later, in 1980, Turner Whitted introduced the well-known ray tracing model "Whitted-Raytracing" [Whi80]. If a ray hits a surface, additional rays are cast for reflection, refraction, and shadow. In the 1990s, ray tracing started to be used in visual effects for movies like "Terminator 2: Judgment Day" (1991) or "Jurassic Park" (1993). In the 2010s, animation studios like Pixar moved to fully path-traced rendering [Chr+18], which uses ray tracing to trace the whole path from a light source to the camera to generate photorealistic images. An example for this is Pixar's animated movie "Finding Dory" (2016) [HVH16].

In 2018, Nvidia paved the way for ray tracing in real-time applications by introducing the RTX Series. This series of graphics cards used the "Turing architecture", which introduced hardware-accelerated ray tracing with the new RT Cores. RT Cores traverse the BVH

(Bounding Volume Hierarchy) autonomously, and by accelerating traversal and ray/triangle intersection tests, they offload the SM (Streaming Multiprocessor), allowing it to handle other vertex, pixel, and compute shading work [Cor18].

Testing every ray against every triangle for intersection is not feasible and takes way too much processing power, even for non-real-time applications. A much faster way to find the triangles that are hit by a ray is to use a bounding volume hierarchy (BVH). It is a tree structure of volumes (usually AABBs), which divides the scene into smaller and smaller subsections as we go down the tree. We can prune a branch as soon as we realize that the section does not intersect with the ray and therefore avoid a lot of ray/triangle intersections.

In this thesis, we will implement hardware-accelerated ray tracing for dynamic environments using the graphics API Vulkan. We will learn how to create the raytracing pipeline, set up our shader binding table, and build and update our acceleration structures. Then, we will test different pipelines and update strategies and other parameters in dynamic scenes containing moving and deforming objects.

2 Related Work

2.1 Ray Tracing

As mentioned in the introduction Appel [App68] and Whitted [Whi80] laid the foundations for ray tracing. For a long time, GPUs were not feasible for ray tracing, and because of that, efficient ray tracing algorithms were developed for the CPU. Möller et al. [MT05] present a clean algorithm for determining whether a ray intersects a triangle. One advantage of this method is that the plane equation need not be computed on the fly nor be stored, which can amount to significant memory savings for triangle meshes. Similarly, another paper presents and efficient and robust algorithm for intersections between rays and AABBs [Wil+05], which is very useful for the bounding volume hierarchies used for acceleration structures. Wald et al. [Wal+14] describe Embree, an open source ray tracing framework for x86 CPUs, which is explicitly designed to achieve high performance in professional rendering environments in which complex geometry and incoherent ray distributions are common.

Later Nvidia introduced the Turing Architecture, which allows for hardware-accelerated raytracing. Using new hardware-based accelerators and a Hybrid Rendering approach, Turing fuses rasterization, real-time ray tracing, AI, and simulation to enable incredible realism in PC games, amazing new effects powered by neural networks, cinematic-quality interactive experiences, and fluid interactivity when creating or navigating complex 3D models [Cor18]. Huerta et al. [Hue+25] analyse these modern GPU architectures by reverse engineering modern NVIDIA GPU cores, unveiling many key aspects of their design.

2.2 Acceleration Structures

There has been a lot of research done on ray tracing acceleration structures. Many of these papers propose new ways of constructing a BVH and discuss the advantages and disadvangtages. One of these methods is KD-tree construction. Soupikov et al. [SK07] presents a highly parallel, linearly scalable technique of kd-tree construction for ray tracing of dynamic geometry. Another paper proposes modifications to previous kd-tree construction algorithms that significantly increase the coherence of memory accesses during construction of the kd-tree [Pop+06]. These and other optimizations are then applied to a SAH (Surface Area Heuristic)-based BVH instead of a KD-tree by Wald [Wal07]. SAH tries to minimize the surface area of the bounding volumes when splitting a parent volume.

Another type of construction is LBVH (Linear Bounding Volume Hierarchy), which uses spatial Morton codes to reduce the BVH construction problem to a simple sorting problem [Lau+09]. There is also another variant of the algorithm called HLBVH (Hierarchical LBVH).

It is a novel hierarchical algorithm that reduces substantially both the amount of computations and the memory traffic required to build a Linear Bounding Volume Hierarchy (LBVH), while still exposing a data-parallel structure [PL10]. Karras et al. [KA13] propose a new massively parallel algorithm for constructing high-quality bounding volume hierarchies (BVHs) for ray tracing, which is based on modifying an existing BVH to improve its quality, and executes in linear time.

A big topic in dynamic environments is updating the acceleration structure. Wald et al. [WBS07] present three algorithmic changes to the typical BVH algorithm to benefit dynamic scenes. First, the BVH is built using a variant of the surface area heuristic conventionally used to build kd-trees. Second, the topology of the BVH is not changed over time so that only the bounding volumes need to be refit from frame-to-frame. Third, and most importantly, packets of rays are traced together through the BVH using a novel integrated packet-frustum traversal scheme. Later, Kopta et al. [Kop+12] describe a method to efficiently extend refitting for animated scenes with tree rotations, a technique previously proposed for off-line improvement of BVH quality for static scenes.

2.3 Skinning Techniques

Since our application uses skeletal animation, we also need to consider different skinning methods. In our application, we use LBS (Linear Blend Skinning), which is a very popular skinning technique due to its simplicity and efficiency. However, LBS does have some issues (e.g., collapsing-joint artifacts), and some research addresses these issues and proposes better alternatives.

Kavan et al. [KŽ05] introduce a new method called SBS (Spherical Blend Skinning). It uses quaternions to interpolate rotations instead of linearly blending transformed vertex positions. A significant advantage of this method is that the input data for this algorithm is the same as for LBS. Another paper presents a novel GPU-friendly skinning algorithm based on dual quaternions [Kav+07]. This approach also solves the artifacts of linear blend skinning at minimal additional cost.

2.4 Summary and Research Gap

We have now reviewed a lot of research regarding ray tracing in dynamic environments. However, the presented research on acceleration structures does not include scenes that use a scene graph or skeletal animation. The test scenes in these papers usually use a single acceleration structure for the whole scene, which gets updated every frame.

Vulkan uses a two-level hierarchy for its acceleration structure, which allows us only to update parts of the scene's acceleration structure. With this thesis, we want to explore real-time raytracing in dynamic scenes using a scene graph and skeletal animation, not only because we want to see how a scene graph takes advantage of Vulkan's two-level acceleration structure, but also because scene graphs and skeletal animation are very common in real-time applications, like video games.

3 Technologies

3.1 System Specifications

Operating System	Ubuntu 24.04.2 LTS
CPU	Intel(R) Xeon(R) W-2235 CPU @ 3.80GHz
GPU	NVIDIA GeForce RTX 3090

Table 3.1: System Specifications

3.2 Build Tools

The project uses **Premake 5**, which does not build the project directly. It is a command-line utility that reads a scripted definition of a software project and generates project files for toolsets like Visual Studio, Xcode, or GNU Make. It is a simpler alternative to CMake.

We use Premake to create **Makefiles**, which then ultimately compile our project.

3.3 Graphics API

As our project's graphics API, we chose **Vulkan 1.2**. Vulkan is a modern platform-agnostic API that handles GPU operations. It is the successor to OpenGL, which was discontinued in 2016. Vulkan allows for much more fine-grained control over the GPU, supports multi-threading, and most importantly for this project, raytracing, through extensions like `VK_KHR_ray_tracing_pipeline` and `VK_KHR_acceleration_structure`. Since these extensions were introduced in Vulkan 1.2, that is the version used in the project.

3.4 Programming Language

At the start of the project, Rust was considered for the project's programming language. Rust is a new, memory-safe, low-level programming language and a modern alternative to C and C++. While Rust has many advantages over C++, we chose **C++** in the end. The main reason is that the Vulkan API was designed for C/C++. There are a lot of great Vulkan libraries for Rust, like the popular "Ash" crate, which is generated from "vk.xml". However, the interface differs from the classic C API in some parts. That could be problematic, considering that nearly all the few resources relating to raytracing in Vulkan use C/C++. That could have

made it challenging to translate the raytracing code into Rust later down the line, so C++ was the safer choice.

3.5 File Formats

3.5.1 glTF

The dynamic 3D scenes that the project uses are stored in the **glTF-2.0** [Khr21] file format. It stores its models in a scene graph and supports skeletal animation and morph animations. By default, it stores the scene data in the popular JSON format. Additionally, buffers are stored in binary BIN files, while textures are stored in image files (e.g., PNG, JPEG, . . .). Since the file format was developed by Khronos Group, which is also behind Vulkan, the data inside the buffers is laid out in a way that makes it easy to use in Vulkan.

3.5.2 CSV

CSV [Sha05] stands for Comma-Separated Values and is used to store tabular data. It is a text-based format and, therefore, very easy to use. Each line of text inside a CSV file represents a row, and the values of that row are separated by commas. We use it to store our benchmarks in our projects. We can later import a CSV file into an application like Excel to create graphs and analyse our gathered data.

3.6 Libraries

3.6.1 volk

volk [zeu] is a meta-loader for Vulkan. It allows you to dynamically load entry points required to use Vulkan without linking to vulkan-1.dll or statically linking Vulkan loader. Additionally, volk simplifies the use of Vulkan extensions by automatically loading all associated entry points.

3.6.2 VMA (Vulkan Memory Allocator)

Memory Management in Vulkan is quite tricky. For example, when a buffer or image is created in Vulkan, no memory is associated with it. The programmer must manually allocate a block of memory with a specific type from a particular memory heap. The properties of these memory heaps depend on the underlying physical device and need to be queried first. Additionally, creating a new memory allocation for each resource may be suboptimal, since the number of allocations can be very limited and many allocations may negatively impact performance. **VMA (Vulkan Memory Allocator)** [GPU] simplifies, automates, and optimizes memory management in Vulkan.

3.6.3 vk-bootstrap

Initializing Vulkan involves a lot of boilerplate code, which will be basically the same in every Vulkan application. **vk-bootstrap** [Lun] can reduce the amount of boilerplate code in a project.

This library simplifies the process of:

- Instance creation
- Physical Device selection
- Device creation
- Queue aquisition
- Swapchain creation

3.6.4 cgltf

cgltf is used to load a glTF file in the project. It parses the glTF file, which is stored in JSON. It would also have been possible to read the JSON directly, but cgltf validates the file, loads the required buffers from their BIN files into memory, and creates a `cgltf_data` struct, which is a lot more comfortable to use. Our project does not use this structure directly, but instead uses it to generate a `Scene` object.

3.6.5 CLI11

CLI11 is a header-only library written in modern C++ designed to parse your application's command-line options. It is simple, feature-rich, and safe. CLI11 allows you to define flags, options, commands, and subcommands. You can also add restrictions to parameters, like only allowing positive numbers or ensuring that a parameter is an existing path.

4 Implementation

4.1 Vulkan Abstraction

4.1.1 Vulkan Handles and Move Semantics

As already mentioned in the Methods and Technology chapter, Rust was originally considered as programming language, because it is modern and memory safe. One of Rust's core features is ownership, which means that each value/object can only be owned and managed by a single variable, and when that variable goes out of scope, it alone is responsible for cleaning up the object. When that variable gets assigned to another variable, the new variable becomes the owner (i.e. the value gets moved). Ownership would be a very nice feature to manage Vulkan objects. Vulkan objects should not be copied on assignment but only moved and when the object is no longer needed (i.e. the owner goes out of scope), it should automatically be destroyed.

While C++ does not support ownership directly, it does have so-called move semantics, with which we can emulate ownership. The ownership of Vulkan handles is managed by the `Handle<T>` class. To do this the class declares a default constructor, move constructor and destructor and overrides the move assignment operator. It is also important, that the copy constructor and copy assignment operator are deleted. The reason for that is that copying a Vulkan object is usually very expensive or not even possible and it is almost never necessary. When we want to copy an object, we need to explicitly create a new object first, before copying the contents of the other object into it (for example using `Buffer::copy_into`).

4.1.2 Shared Pointers

Since C++11 the language supports so-called smart pointers, which make memory management safer and less complicated. One of these smart pointers is `std::shared_ptr<T>` (or our alias `ptr::Shared<T>`). This type of pointer allocates an object, which can then be shared between multiple owners. The object is alive while it has at least one owner, and the last owner, that goes out of scope, cleans up the object. This structure can be combined very nicely with our previously mentioned `Handle<T>`. A Vulkan object can only have one owner, which is a harsh limitation and often causes problems. If we wrap the object in a shared pointer, it can be shared across multiple owners.

4.1.3 The Builder Pattern

As mentioned before, we want the creation of Vulkan objects to be very explicit. The first method to create objects in C++ you think of are constructors, but they prove to be problematic

for a couple of reasons. The first problem is that constructors are often called implicitly (e.g. the default constructor gets called when declaring a variable). Default construction only makes sense for very few Vulkan objects (like for example a VkFence) and we do not want a Vulkan object to be created when implicitly (or explicitly) calling the default constructor. So, for that reason, the default constructor simply does not create a Vulkan object or, in other words, leaves the handle at `VK_NULL_HANDLE`. Now if a different constructor actually creates an object, the behavior of constructors becomes inconsistent regarding Vulkan object creation. In addition to that, Vulkan objects are almost always created using a `Vk*CreateInfo` struct, which may take a lot of parameters. To solve all of these issues we implemented a `*Builder` class for most Vulkan objects. Some exceptions are `Fence` and `ImageView`, which are created with `Fence::create` and `ImageView::create_from` respectively. This makes it very clear when a Vulkan object is created, since it can only be created using a `*Builder` or a static method (like `Fence::create`) and especially it can never be created by a constructor. This is also the reason Vulkan objects never have a constructor, besides the default constructor.

```
// declare the buffer (handle is VK_NULL_HANDLE)
Buffer buffer;

// creating the buffer
buffer = BufferBuilder()
    .add_usage(VK_BUFFER_USAGE_TRANSFER_DST_BIT)
    .add_usage(VK_BUFFER_USAGE_VERTEX_BUFFER_BIT)
    .memory_usage(VMA_MEMORY_USAGE_GPU_ONLY)
    .size(128)
    .build();
```

Figure 4.1: Sample-code of the builder pattern for a buffer

4.1.4 Basic Vulkan Object Structure

All of the Vulkan objects follow a basic structure, which can be seen in the following code sample:

```
class Foo : public utils::Move, public ptr::ToShared<Foo> {
    friend class FooBuilder;
private:
    Handle<VkFoo> foo_handle{};
    // Additional member variables

public:
    Foo() = default;

    inline VkFoo handle() const { return foo_handle; }
    // Additional getter functions

    // Additional functions
};

class FooBuilder {
public:
    using Ref = FooBuilder&;
private:
    // build parameters, e.g. VkFormat _format;

public:
    FooBuilder() = default;

    // parameter setter functions, e.g. Ref format(VkFormat format);

    Foo build() const;
}
```

Figure 4.2: The basic structure of every class representing a Vulkan object

It is important to note, that `Foo` is just a placeholder for an actual Vulkan object (like `Buffer`, `Image` or `Pipeline`). Usually, the name of the class is just the name of the Vulkan handle without the "Vk" prefix (e.g. `VkBuffer` $\implies$ `Buffer`), but there are some exceptions (e.g. a `Shader` class holds a `VkShaderModule` handle). As we can see, the Vulkan handle is wrapped in the `Handle<T>` class, which ensures correct move semantics. The member function `handle` allows us to access the raw Vulkan handle.

The class inherits from `utils::Move` and `ptr::ToShared<Foo>`. The `utils::Move` class implements the default move constructor and move assignment operator, and it deletes the copy constructor and copy assignment operator. The `ptr::ToShared<Foo>` class contains the function `to_shared`, which is just syntax sugar for moving the object into a shared pointer.

The additional variables in the class are often used to store meta-data (e.g the format and extent of an image) or to keep other objects alive in a shared pointer, that are referenced by the underlying object (e.g. the `RenderPass` is referenced by a `Pipeline`).

The may also be additional functions, which actually use or manipulate the underlying object. Often, these functions have a `cmd_` prefix, which means that they are wrappers around one or multiple commands (e.g. `Buffer::cmd_copy_into` is a wrapper for the Vulkan function `vkCmdCopyBuffer`).

Even though not every Vulkan object uses a `*Builder` class, it is also shown in the sample code, since most Vulkan objects have one.

4.1.5 The Vulkan Context

Before we can actually do any work in Vulkan, we need to do a lot of setup. This usually includes the following steps:

1. Create a `VkInstance`.
2. Select a `VkPhysicalDevice`, which supports the extenstions, features and capabilities required by the project.
3. Create a `VkDevice` from the selected `VkPhysicalDevice`.
4. When creating the `VkDevice`, `VkQueue`s from a certain queue family need to be enabled, to be able to submit command buffers later.

These Vulkan objects are accessed very frequently throughout the project. For example, you need to pass the `VkDevice` as a parameter when creating or destroying most Vulkan objects and a command buffer, which executes a sequence of Vulkan commands, needs to be submitted to a certain `VkQueue`. Keeping track of all of these Vulkan objects is quite tideous, which is why we introduced the Vulkan context. It is represented by the `Context` class, which stores these Vulkan objects, allows the programmer to access them anywhere and reduces the effort of setting up Vulkan.

The Vulkan context contains the following objects:

- the `VkInstance`
- the `VkPhysicalDevice` and `VkDevice`
- an optional `GLFWwindow*` from which a `VkSurfaceKHR` is created and also stored (this is necessary for rendering to a window, but it will not be used in our main application, since it is headless)
- the `CommandManager` (manages queues and command pools)
- the `VmaAllocator` (used when allocation memory)

The class is designed similar to a singleton, but it deviates in some parts. A singleton usually creates its instance at the start of the application or when it is used for the first time (lazy instantiation). However, `Context` can not be implicitly instanciated this way, because it needs some parameters stored in a `ContextInfo`, when being instanciated. For this reason we need to explicitly create the Vulkan context using `Context::create(...)` with the desired `ContextInfo` before calling `Context::instance()` for the first time or we will get a `nullptr`. While all other objects in our project get cleaned up automatically when going out of scope, the Vulkan context also needs to be destroyed explicitly using `Context::destroy()` when it is no longer needed. If we now want to get the current `VkDevice`, we can simply call `Context::instance()->get_device()`.

4.2 Raytracing-related Objects

In this section we will take a closer look at all the objects related to ray tracing and our abstractions of them.

4.2.1 Acceleration Structures

We are going to start with the acceleration structures. Acceleration structures are data structures used by the implementation to efficiently manage scene geometry as it is traversed during a ray tracing query [Khr25]. The acceleration structure has two levels:

- **Bottom Level Acceleration Structure (BLAS)**

  The lower level, the bottom-level acceleration structure, contains the triangles or axis-aligned bounding boxes (AABBs) of custom geometry that make up the scene [Koc+20]. This usually corresponds to a mesh, that is used in a scene.

- **Top Level Acceleration Structure (TLAS)**

  The upper level, the top-level acceleration structure, contains references to a set of bottom-level acceleration structures, each reference including shading and transform information for that reference [Koc+20]. This represents a scene filled with objects, that each can have a different transformation and the mesh.

The relationship between TLAS and BLAS can be seen in this figure:

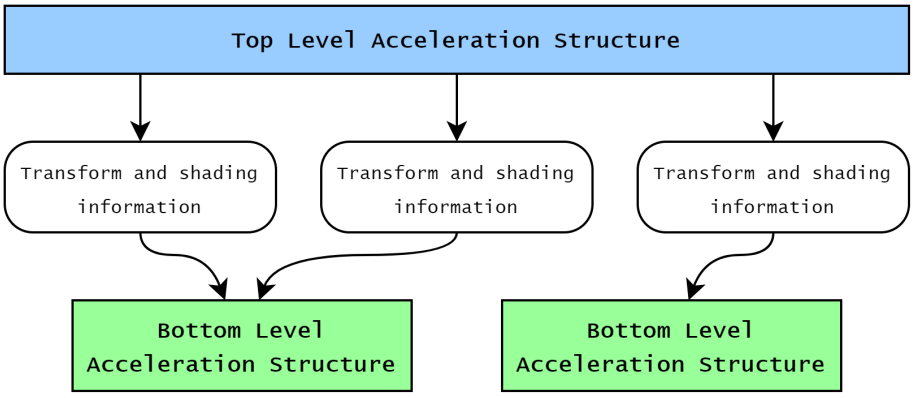


Figure 4.3: Acceleration structure

Vulkan represents both types of with the `VkAccelerationStructureKHR` handle. This is however just a handle with some meta-data and we need a `VkBuffer` to store the actual data of the acceleration structure (e.g. the BVH). Before building an acceleration structure we need to reference the data used for building in a `VkAccelerationStructureGeometryKHR`. For the BLAS this is usually a list of trianges (even though other primitives are also possible)

and for the TLAS it is a list of instances, with transfrom and shading information as well as a handle to a BLAS. Once our `VkAccelerationStructureGeometryKHR` is ready we can query for the `VkAccelerationStructureBuildSizesInfoKHR`, which contains the variable `accelerationStructureSize`, which is the size for the buffer, that stores the acceleration structure, as well as the variables `updateScratchSize` and `buildScratchSize`, which are the required sizes of a temporary scratch buffer which is only used during building/updating the acceleration structure. Now we can build an acceleration structure using the command `vkCmdBuildAccelerationStructuresKHR`.

We encapsule a BLAS and TLAS in the classes `Blas` and `Tlas` respectively. They both have the following member variables:

- `blas` / `tlas`: The `Handle<VkAccelerationStructureKHR>`.
- `buffer`: The buffer that stores the data for the acceleration structure.
- `build_scratch_buffer`: The scratch buffer used during building.
- `update_scratch_buffer`: The scratch buffer used during updating/refitting.

These classes also have a builder (see subsection 4.1.3 and subsection 4.1.4), which expects a list of `BlasGeometry`s when building a `Blas` and a list of `TlasInstance`s when building a `Tlas`. A `BlasGeometry` describes the vertex buffer and optional index buffer used for a triangle mesh. A `TlasInstance` on the other hand stores a reference to a BLAS in a `ptr::Shared<Blas>` and a transformation. The builder also has the property `dynamic`, which needs to be set to `true`, if you want to refit the acceleration structure later. It also has the property `fast_build`, which if set to `true` will set the flag `VK_BUILD_ACCELERATION_STRUCTURE_PREFER_FAST_BUILD_BIT_KHR` instead of `VK_BUILD_ACCELERATION_ST` and prefer a fast build over a fast trace.

When we want to update an acceleration structure, there are two functions we can use:

- `rebuild`:

  This function recreates the BVH from scratch. It essentially does the same thing as the `build` function in the builder, except that it uses the already created handle and buffers and does not create them again.

- `refit`:

  This function does not change the structure of the BVH. Instead it only adjusts the bounds of the AABBs in the BVH for the new vertex positions. This is faster than rebuilding from scratch, but can lead to a sub-optimal BVH, which results in higher ray tracing durations.

4.2.2 Raytracing Pipeline

In order to issue ray tracing commands we need a ray tracing pipeline. The following figure shows a flow chart of the ray tracing pipeline.

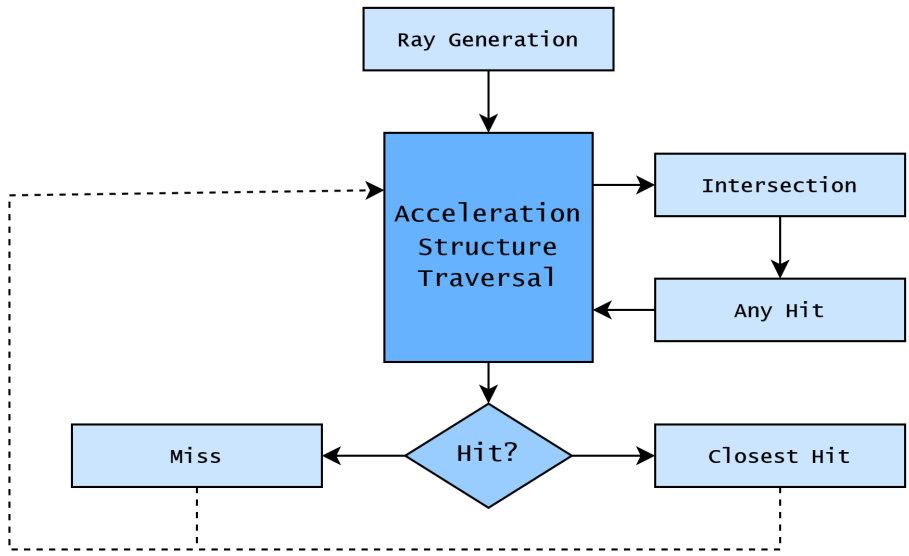


Figure 4.4: Ray tracing pipeline

A raytracing pipeline in Vulkan stores a list of shader groups. A shader group consists of one or more shaders and in our application we are only interested in two types of shader groups:

- **General Shader Group**

  A general shader group only stores a single "general" shader, which it can call independently of any ray/triangle intersections or hits. We use this type of shader group in the ray generation and miss stage of the pipeline.

- **Triangle Hit Shader Group**

  A triangle hit shader group contains the any-hit shader and the closest-hit shader, which can be called when a ray/triangle intersection occurs. It calls the any-hit shader for every ray/triangle intersection along the ray and the closest-hit shader only for the first intersections, if the ray intersects a triangle at least once.

The class `ShaderGroup` implements shader groups in our application. Each `ShaderGroup` contains a unique identifier, which can be accessed using the function `id`. It contains either a `ShaderGroupGeneral` or a `ShaderGroupHit` struct inside a `std::variant`. A `ShaderGroupGeneral` contains the shader `general`, while a `ShaderGroupHit` contains the shaders `closest_hit` and `any_hit`.

The raytracing pipeline is contained in the class `RtxPipeline`, which is created from a list of `ShaderGroup`s. The `ShaderGroup` objects are just descriptions of the shader groups. The actual shader groups get allocated by the raytracing pipeline and can be accessed through an array of shader group handles. The `RtxPipeline` stores a map from the `id` of every `ShaderGroup` to the index of the shader group handles, to access the handles easier.

The raytracing pipeline also needs a pipeline layout, which is stored in a `PipelineLayout` object and it needs to set the maximum recursion depth of rays in `max_ray_recursion_depth`.

4.2.3 Shader Binding Table

Once the ray tracing pipeline is ready we need to create a shader binding table (SBT), which selects the correct shader group in every stage of the raytracing pipeline.

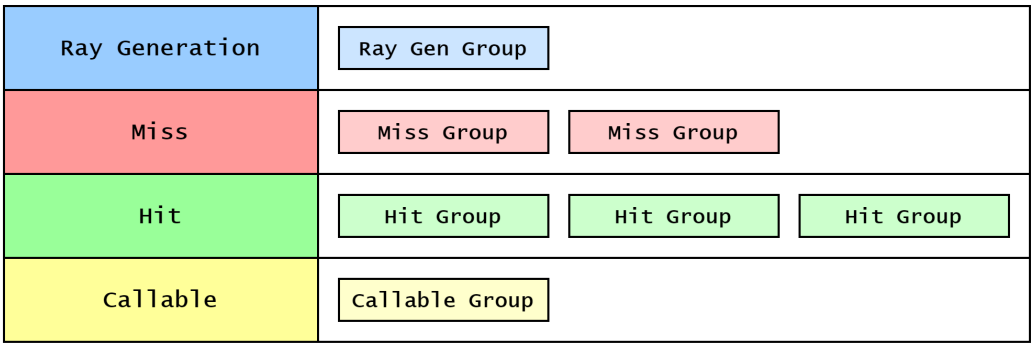


Figure 4.5: Shader binding table

As we can see it has 4 rows:

- **Ray Generation**: Shader group used for ray generation.
- **Miss**: Shader groups that can be used when a ray misses.
- **Hit**: Shader groups that can be used when a ray intersects with at least one object.
- **Callable**: Shader groups, which contain arbitrary shaders that can be called by other shaders and are not tied to a specific ray traversal event. (Not used in our application)

The shader binding table needs to be stored in a `VkBuffer` and the rows of the table are represented by a `VkStridedDeviceAddressRegionKHR`, which references a region inside that buffer. A `VkStridedDeviceAddressRegionKHR` also contains a stride, which specifies how many bytes we need to get from one element to the next in that region.

When casting a ray in a shader using the function `traceRayEXT`, we can specify a miss offset, which is the offset of the shader group in the miss row, that should be used when the ray misses. In a similar way we can also define an offset into the hit shader groups row, to specify which shader group our ray should use when a hit occurs. Every instance in the TLAS can also define a an additional offset into the hit shader group, which gets added to the previously mentioned offset.

The SBT is contained in the `SBT` class. Because the SBT is strongy dependend on the raytracing pipeline, `SBT` does not use a builder, but rather we build it with the member function `build_sbt` inside `RtxPipeline` and a `SBTInfo` struct as a parameter. `SBTInfo` has the following member variables:

- `ray_gen_group`: The `ShaderGroup` used for ray generation.
- `miss_groups`: A list of `ShaderGroup`s, which can be used when a ray misses.
- `hit_groups`: A list of `ShaderGroup`s, which can be used when a ray hits.

The `ShaderGroup`s used inside the `SBTInfo` must be a sub-set of the `ShaderGroup`s used to create the ray tracing pipeline. When building the SBT we first retrieve the array of shader group handles of the ray tracing pipeline using `vkGetRayTracingShaderGroupHandlesKHR` and with the mapping from shader group ID to index we can get the index of the handle for every shader group. We then store the ray generation shader group handle, followed by the sequence of miss shader group handles and finally the sequence of hit shader group handles in the buffer of the `SBT`. Finally, we set the `VkStridedDeviceAddressRegionKHR` for every row and our `SBT` is completed. We can now use our table rows when calling `vkCmdTraceRaysKHR`.

In the following code sample we can see how to create the ray tracing pipeline and SBT:

```
// creating shader groups from shaders
ShaderGroup ray_gen_group = ShaderGroup::create_general(ray_gen_shader);
ShaderGroup miss_group = ShaderGroup::create_general(miss_shader);
...
ShaderGroup hit_group = ShaderGroup::create_hit_closest(closest_hit_shader);
...

//creating the ray tracing pipeline
RtxPipeline pipeline = RtxPipelineBuilder()
    .add_shader_group(ray_gen_group)
    .add_shader_group(miss_group)
    .add_shader_group(hit_group)
        ...
    .layout(pipeline_layout)
    .max_ray_recursion_depth(1)
    .build();

//createing the SBT
SBT sbt = pipeline.build_sbt(SBTInfo()
    .ray_gen_group(ray_gen_group) // ray gen row
    .miss_groups({ miss_group, ... }) // miss row
    .hit_groups({ hit_group, ... })); // hit row
```

Figure 4.6: Sample-code to create ray tracing pipeline and SBT

4.3 Scene

In this section we are looking at the structure and behaviour of our implementation of a scene. In order to understand, why we structured our scenes the way we did, we first need to take a look at the glTF file format, from which our scenes are loaded from.

4.3.1 glTF Structure

A glTF file stores its data in the JSON format. There are different types of objects in glTF and each of them is stored in an array. An objects can reference an other object by its index in the respective array. The objects we are interested in are:

- **Buffer, Buffer view and Accessor**

  These three objects work together closely. A buffer stores a block of binary data, usually contained in a separate BIN file. A buffer view describes a certain part of a buffer. It references a certain buffer and contains an offset, length and optional stride. A buffer view sees the data as just raw bytes. To associate some kind of meaning to the data we use an accessor. It references a buffer view with a certain offset and contains the number of elements, as well as the type (e.g. scalar, vec3, mat4, ...) and component type (e.g. float, unsigned int, ...) of the elements. Accessors can be used for things like vertex attributes and joint matrices.

- **Mesh**

  A mesh stores a static 3D model. It contains a material and an array of primitives. Each primitive has a mode (e.g. points, lines, triangles), an optional accessor for indices and a set of accessors used for the different vertex attributes (e.g. positions, normals, uv coordiantes, vertex colors, joint weights).

- **Skin**

  A skin is used for skeletal animation of a mesh. It contains an array of nodes, that are used as the skeletons joints, and an accessor for the inverse bind matrices of the joints.

- **Node**

  This represents a node in the scene graph. A node always has a transform, which is either represented by a 4x4 matrix or a scale (vec3), rotation (quaternion) and translation (vec3). A node can have an array of child nodes, which inherit the transform of the parent node. In other words, a nodes transform is applied in the parent nodes coordinate system.

  A node can optionally also contain a mesh and, if a mesh is present, also a skin. The skin can deform the node's mesh during an animation.

- **Scene**

  A scene contains an array of root nodes. These nodes and all of their children are part of that scene.

- **Animation**

  An animation has an array of channels and an array of samplers. A channel describes, which property is being sampled during the animation. It contains the effected node and a path for the effected property of the node (e.g. "transform", "scale", "rotation"). It also holds a reference to one of the animations samplers.

  A sampler is used to calculate the value of a property at a certain point in time from a set of key frames. It contains two accessors, one for the time of the key frames and one for the value of the keyframes. Additionally, a sampler contains the type of interpolation, that should be used to sample a value.

There are also some other objects (e.g. materials, textures and cameras), but since they are not relevant in our application, we don't explain them here.

4.3.2 Scene Structure

We will now take a look at the structure of our scene implementation. For this we will look at the C++ classes we used to implement the different glTF objects.

The `Scene` class

The `Scene` class represents a scene that can be loaded from a glTF file. It contains an array of nodes, an array of animations and an array of skins. Additionally, it stores the TLAS, that is used for the ray traversal through the scene, and the different buffers used by the scene in a `SceneBuffers` struct.

A scene can be loaded using the static method `Scene::load`, with the path to the glTF file as parameter. A scene also contains the member function `Scene::iter`, which returns a node iterator for all of the nodes contained in the scene. There also is the function `Scene::build_acceleration_structures`, which we need to call on a scene to build all of the necessary acceleration structures so we can use ray tracing. The rest of the scenes functions are explained in the next subsection about updating a scene (see subsection 4.3.3).

The `Mesh` class

The `Mesh` class represents a glTF mesh. It stores an array of `Primitive`s. Each `Primitive` stores three sub-buffers: `positions` for the vertex positions, `indices` for the vertex indices and `joint_weights` for the joint weights used for skinning the mesh. The positions and indices are pretty straight forward and just store a sequence of `vec3`s and `unsigned int`s respectively. The joint weights are a bit more complex, because they are stored in a 2D array of `JointWeight` structs. A `JointWeight` contains the `index` of the node in the skeleton of the skin and its `weight`. All the arrays of joint weights for a vertex are stored in sequence like shown in this figure:

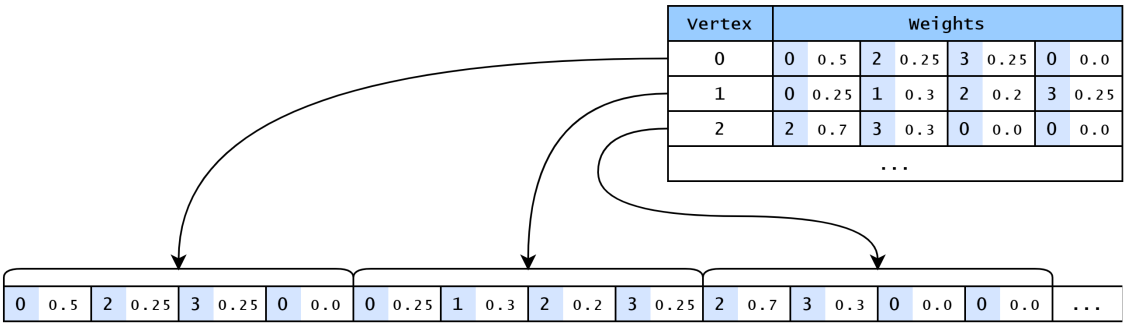


Figure 4.7: Joint weight layout

Additionally, each `Primitive` also contains the member variables `positions_cpu` and `joint_weights_cpu`, which store the same data as `positions` and `joint_weights`, but this time inside `std::vector`s for easier access in CPU skinning.

Finally, a `Mesh` can also contain a `Blas`, which is created for the static mesh. It is only created if a mesh is used statically by a node, i.e. a node that contains the mesh without a skin.

The `Skin` class

The `Skin` class is relatively simple and matches the glTF skin object almost exactly. It simply stores an array of `Node`s, which represent the joints in the skeleton and an array for the inverse bind matrices, with one entry for each joint.

The `Node` class

The `Node` class directly represents a node in glTF. As we mentioned previously the transform of a node can either be stored as a matrix or separated into a translation, rotation and scale. To model that in our application, we added the struct `RawTransform`, which contains the transform, rotation and scale. We also have the struct `NodeTransform`, which stores the local transform of a node and looks like this:

```
struct NodeTransform {
    bool raw;
    union {
        glm::mat4 matrix;
        RawTransform raw;
    } transform;
}
```

Figure 4.8: Structure used to store local transform of a node

The union `transform` is used to store either a `glm::mat4` or a `RawTransform`. The boolean `raw` is used to determine, how the union should be interpreted, i.e. if `raw` is `true`, the union contains a raw transform, otherwise it contains a matrix. The node also stores the global transform as a 4x4 matrix, but it is only valid after calling `Node::update_global_transform`.

A node also stores an array of child nodes and it can also contain a `Mesh`. If a mesh is present, it also stores a `Blas` and the node must generate a `TlasInstance` that references the node's `Blas` when building the acceleration structure. After building the acceleration structure for the first time, the node also stores the index of the `TlasInstance`, which is necessary to update the `TlasInstance` later. If the node does not have a `Skin`, the node's `Blas` is the the same as the one inside the mesh.

If the node does however contain a `Skin`, it stores an array of sub-buffers for dynamic positions. For every primitive of the node's `Mesh`, a sub-buffer for dynamic positions needs to be present in the array. Each sub-buffer needs to have the same size as the sub-buffer for the static positions of the respective primitive. These `dynamic_positions` are used to store the vertex positions of a mesh after skinning. If a `Skin` is present, the `Blas` is not simply just a reference the meshes' `Blas`, but it is a separate, dynamic `Blas`, which can be updated with the node's `dynamic_positions`.

The `Animation` class

The `Animation` class represents a glTF animation. It contains an array of `AnimationSampler`s, which contain a reference to the node, that they modify during animation, and also the virtual function `apply_for`. This function samples a value at the specified point in time and applies it to a certain property of the nodes transform. There are three sub-classes of `AnimationSampler`, which override the `apply_for` function:

- `TranslationSampler`: samples a 3D vector and sets it as the nodes translation.
- `RotationSampler`: samples a quaternion and sets it as the nodes rotation.
- `ScaleSampler`: samples a 3D vector and sets it as the nodes scale.

The animation itself also has an `apply_for` function, which iterates over all animation channels and applies them for the given point in time.

Buffers and Sub-Buffers

glTF uses buffers, buffer views and accessors. We dont use the data in the glTF buffers directly, but we read the data and lay it out in a more convenient way. A glTF accessor may access data, that is interleaved with other data, or sparse data. Additionally, the accessor's component type may not match our component type. However, we want to have our data lined up in memory sequentially and of a fixed type. Luckily, the cgltf library offers the function `cgltf_accessor_unpack_floats`. This function reads all the values of the accessor, converts the components of the values to `float` and stores the values sequentially in a target buffer. The function `cgltf_accessor_unpack_indices` does the equivalent for `unsigned int`s.

We create one buffer for each type of data: one for vertex indices, one for joint weights, one for static vertex positions and one for dynamic vertex positions. An attribute can then reference a sub-subsection into one of these buffers using a sub-buffer. A sub-buffer has a reference to the buffer of intrest using a `ptr::Shared<Buffer>`, as well as an offset and a length. We do not need to store the type of the elements in the sub-buffer, since the type is fixed and can be determined by the variable. For example, we know that the sub-buffer `positions` in a `Mesh` stores a sequence of `vec3`s.

The following figure gives a little bit more insight, of how these sub-buffers are used by meshes and nodes.

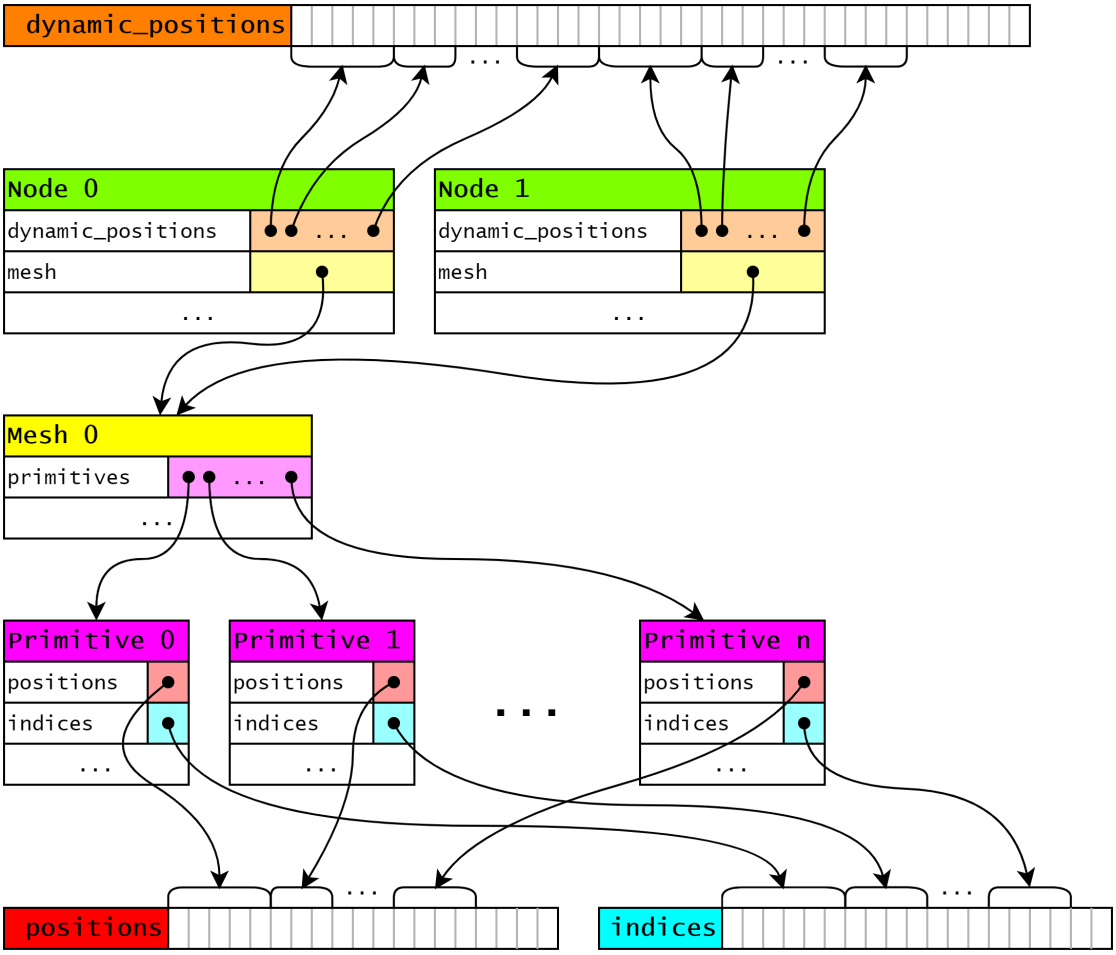


Figure 4.9: Usage of sub-buffers in a scene

In this graphic we can see, how both nodes `Node 0` and `Node 1` reference the same mesh `Mesh 0`. This means, that they use the same sets of static positions and indices. But because two nodes with the same mesh can use different skins, each node needs to store their own set

of dynamic positions. We can also see how the size of every `dynamic_positions` sub-buffer matches the size of the respective sub-buffer in `positions`.

4.3.3 Updating a Scene

Since our scenes are dynamic, we need a way to update them. We need update our scene after calling `Animation::apply_for` from an animation in our scene. A scene is updated in three steps:

1. updating the global transform for every node
2. skinning all animated models
3. updating the acceleration structure

When we want to update our scene we need to perform all of the above steps in the exact order. We keep the steps separate however, because we can use different implementations for some steps. For example, we can skin our models on the CPU or on the GPU and we can update our acceleration structure by either rebuilding it or refitting it.

Updating Global Transform

Using the member function `Node::update_global_transform` we can update the global transform of a node. This function takes in a `glm::mat4` as argument, which represents the parents transform. This argument is optional and gets set to the identity matrix by default. The following pseudo code shows the functions behaviour:

Algorithm 1 Pseudo code for updating the global transform

```
1: function UPDATE_GLOBAL_TRANSFORM(node, parent_transform)
2:     node.global_transform ← parent_transform × node.transform
3:     for all child in node.children do
4:         update_global_transform(child, global_transform)
5:     end for
6: end function
```

The global transform is calculated by pre-multiplying the parents transform with the local transform of the node. We then call `Node::update_global_transform` recursively for every child node, with the new global transform of the current node as a parameter.

Skinning

Skinning describes the process of applying the deformation of a skeleton to a certain model. To do that we first need to calculate the joint matrices:

Algorithm 2 Calculating joint matrices

```
for i ← 0 to joints.count - 1 do
    joint_matrices[i] = inverse(node.global_transform)
      × joints[i].global_transform
      × inverse_bind_matrices[i]
end for
```

After having calculated the joint matrices, we can skin the model:

Algorithm 3 Skinning a model

```
for j ← 0 to (positions.count −1) do
    matrix ← 0_{4×4}
    for all joint_weight in joint_weights[i] do
        matrix ← matrix + joint_weight.weight × joint_matrices[joint_weight.index]
    end for
    position ← matrix × position
    dynamic_positions[i] ← matrix × positions[i]
end for
```

This algorithm uses **Linear Blend Skinning**, which is the standard skinning technique to its simplicity and efficiency [Jac+12]. We basically create a weighted sum of all the joint matrices used by the certain vertex and then multiply the resulting matrix with the vertex position to get the skinned position.

We have two implementations of the above skinning algorithm:

- **CPU skinning**

  This implementation read all the vertex positions from the sub-buffer, runs the algorithm for each of them and stores the results in the respective sub-buffer for dynamic positions.

- **GPU skinning**

  This implementation uses a vertex shader to run the algorithm for every vertex position and stores the resulting dynamic positions in a storage buffer. The pipeline that contains the vertex shader also disables rasterization, because no fragments should be generated during drawing and the pipeline should stop after running the vertex shader.

  We could have also used a compute shader, which may perform better, but then we would need to implement a compute pipeline. With the approach we chose, we can simply repurpose the rasterization pipeline, which was already implemented.

Updating the acceleration structure

Algorithm 4 Updating acceleration structure

```
1:  function UPDATE_ACCELERATION_STRUCTURE(scene)
2:      node_iterator = get_node_iterator(scene)
3:      tlas_instances = get_instances(scene.tlas)
4:      for all node in node_iterator do
5:          if has_blas(node) then
6:              if is_dynamic(node.blas) then
7:                  update(node.blas)
8:              end if
9:              update_tlas_instance(tlas, node)
10:         end if
11:     end for
12:     update(scene.tlas)
13: end function
```

There is no `Scene::update_acceleration_structures` function, that you can call to update your acceleration structures. Instead, there are two slightly different member functions you can choose:

- `Scene::rebuild_acceleration_structures`
- `Scene::refit_acceleration_structures`

They both run more or less the same algorithm, the only difference is, that one uses "rebuilding" to update its acceleration structures and the other one uses "refitting".

4.4 Application Overview

In this section we will take a look at what our application does exactly. We can start our application with some options:

- **Test Scene**: The scene, from a set of test scenes, that we want to run.
- **Pipeline**: The pipeline we want to use to render the scene (see subsection 4.4.1).
- **Frame Count**: The number of frames to be rendered.
- **Resolution**: The resolution of our frames.
- **SPP (Samples per Pixel)**: Number of rays cast per pixel (see subsection 4.4.2).
- **Delta Time**: The amount of time the animation should move between frames.
- **Rebuild Frequency**: Number of frames, after which a rebuild of the scene is done.
- **Fast Build vs Fast Trace**: Either prefers fast builds or fast ray tracing.

At the start, the application creates the desired ray tracing pipeline and loads the specified test scenes. It then applies the scenes animation for the current time, updates the nodes global transforms, skins our meshes and updates the scenes acceleration structures. After that, it traces the rays and advances the animation time by the delta time. These steps are repeated for the specified number of frames.

The application tracks the duration of acceleration structure updates, tracing of rays and their combined time for each frame. At the end, the application stores list of tracked times, together with their averages, in a CSV file.

4.4.1 Pipelines

There are three possible pipelines:

- **Basic Pipeline**

  This pipeline is a very basic, which only casts primary rays. If a primary ray hits any object, it returns the color white, otherwise it will returns the color black.

- **Standard Pipeline**

  This pipeline is an extension of the basic pipeline. Its shaders have access to a directional light source and the scene's geometry. Now when a ray hits, we can use this information to apply some simple shading.

- **Shadow Pipeline**

  This pipeline is an extension of the standard pipeline, which uses secondary rays. When a primary ray hits a surface, a shadow ray is cast from the surface point towards the light. This ray checks, if the path to the light is unobstructed, before shading the sample.

4.4.2 Multi-Sampling

Our application also supports multi-sampling, which is a common method to do anti-aliasing. We also use multi-sampling in our application to increase the number of rays in our test scenes without having to use gigantic images.

We use a very simple method, where we devide each pixel into an $n \times n$ grid. This means the number of samples per pixel needs to be a square number. We then cast one ray through the center of every grid cell and take the average of the samples to determine the pixels final color.

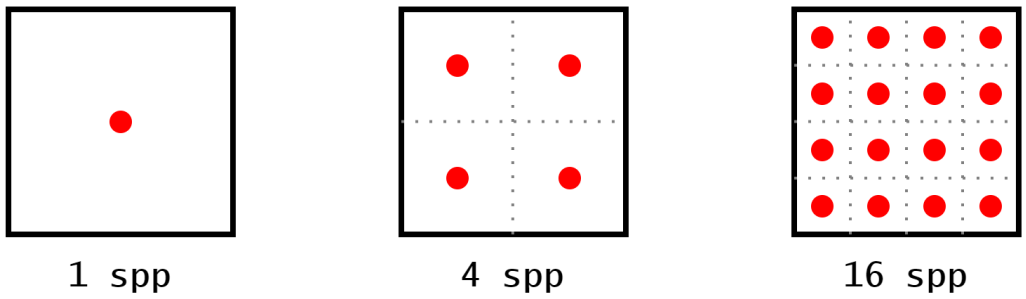


Figure 4.10: Multi-sampling grid

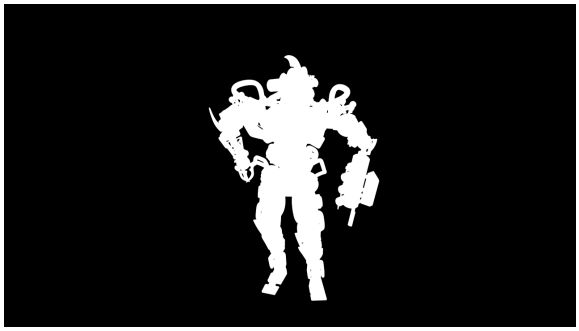
5 Results

5.1 Pipelines

5.1.1 Output

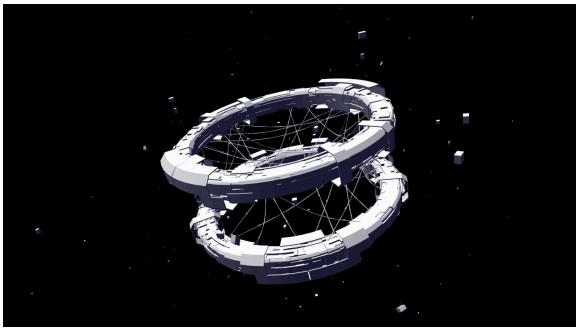
(a) Standard pipeline

(b) Shadow pipeline

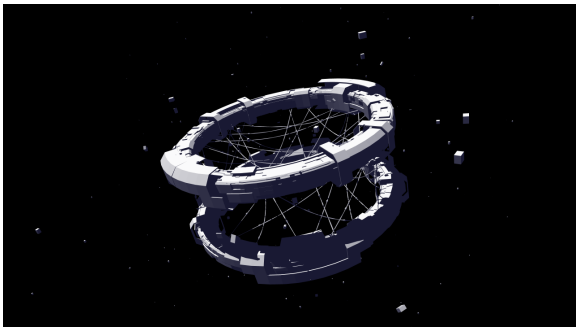


(c) Basic pipeline

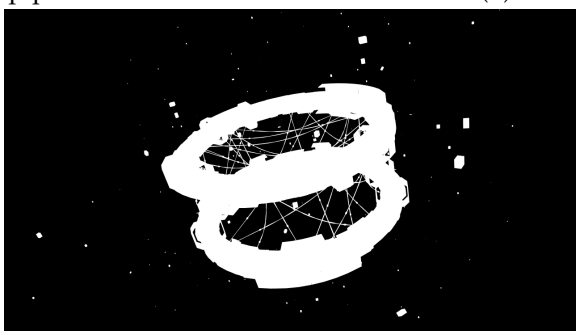
Figure 5.1: Output images using different pipelines for the scene "BrainStem"



(a) Standard pipeline

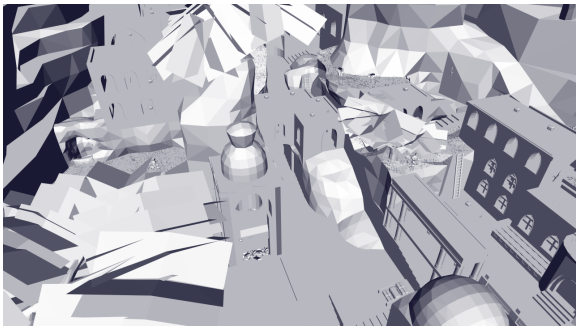


(b) Shadow pipeline

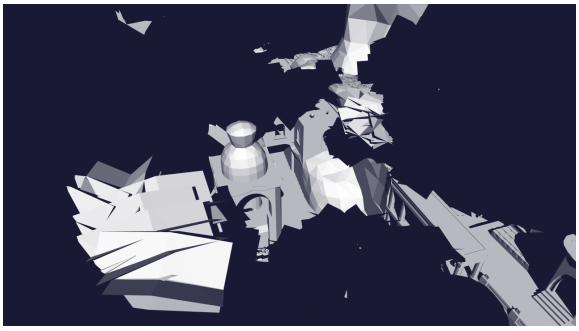


(c) Basic pipeline

Figure 5.2: Output images using different pipelines for the scene "Space Station"



(a) Standard pipeline



(b) Shadow pipeline

Figure 5.3: Output images using different pipelines for the scene "Eternal Valley"

5.1.2 Performace

We tested the different pipelines for the scene "Eternal Valley". We generated 64 frames with a resolution of 1280×720 pixels and 64 samples per pixel. We also rebuilt the acceleration structure every frame and prefered a fast trace for optimal ray tracing durations.

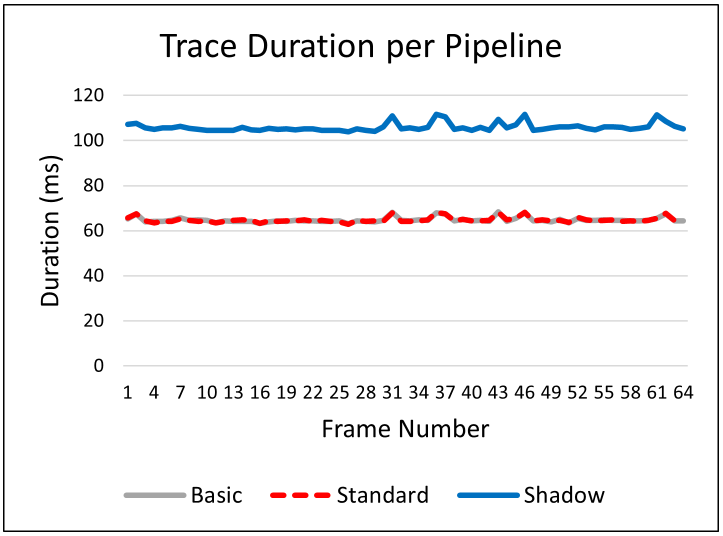


Figure 5.4: Impact of different pipelines on ray tracing duration

Pipeline	Avg. Trace
Basic	64.803 ms
Standard	64.802 ms
Shadow	105.898 ms

Table 5.1: Average trace duration per pipeline

We can see that the basic pipeline and standard pipeline have basically the same behaviour. This would mean, that our shaders do not have a big impact on the performace of our pipelines and that the large majority of time is spent on the traversal of the acceleration structure.

We can also see that our shadow pipeline takes up a lot more time that the other two. This makes sense, because cast an additional ray for every primary ray that hits a surface. In our test scene, every ray hits a surface (see Figure 5.1.1), which means that every primary ray also produces a shadow ray and the total number of rays in the shadow pipeline is double the number of rays in the other two pipelines. However, the trace duration only increases by about 63% instead of 100%. This has probably to do with the way we cast our shadow rays, because we use two flags that are not used for primary rays:

- `gl_RayFlagsSkipClosestHitShaderEXT`

  This flag prevents the ray from invoking the closest hit shader. We can use it, because our implementation relies entirely on the miss shader. The surface point is assumed to be in shadow (`shadow = true`) and the miss shader then sets `shadow` to `false`.

- `gl_RayFlagsTerminateOnFirstHitEXT`

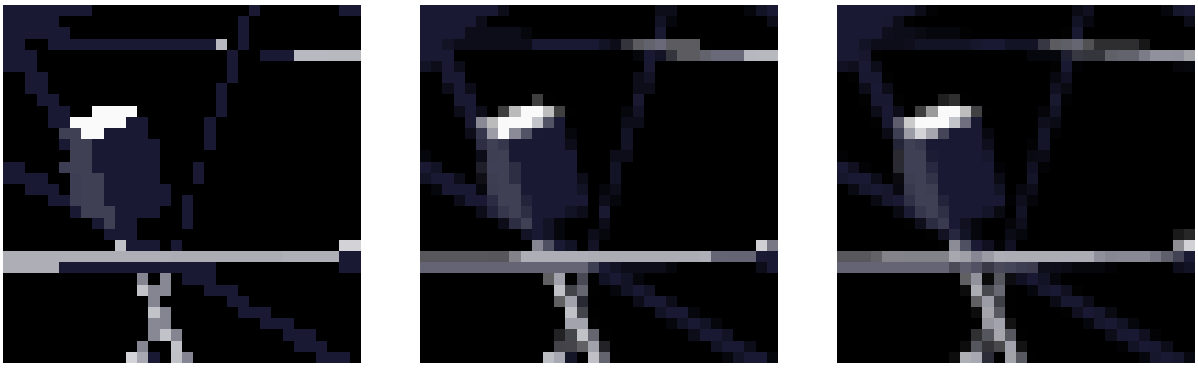
  This flag terminates the traversal of the acceleration structure as soon as anything is hit. We only use the miss shader, which means that as soon as we hit something, the miss shader will not be invoked and we can safely terminate early.

This could explain why shadow rays are cheaper in our implementation.

Another reason could be that, because we use a directional light source, all shadow rays are parallel to eachother, which makes them more coherent and may also improve performance.

5.2 Multi-Sampling

5.2.1 Output



(a) 1 spp (b) 4 spp (c) 16 spp

Figure 5.5: Comparison between images using a different number of samples per pixel

5.2.2 Performace

We tested different numbers of samples per pixel for the scene "Space Station" with the shadow pipeline. We generated 100 frames with a resolution of 1280×720 pixels. We also rebuilt the acceleration structure every frame and prefered a fast trace for optimal ray tracing durations.

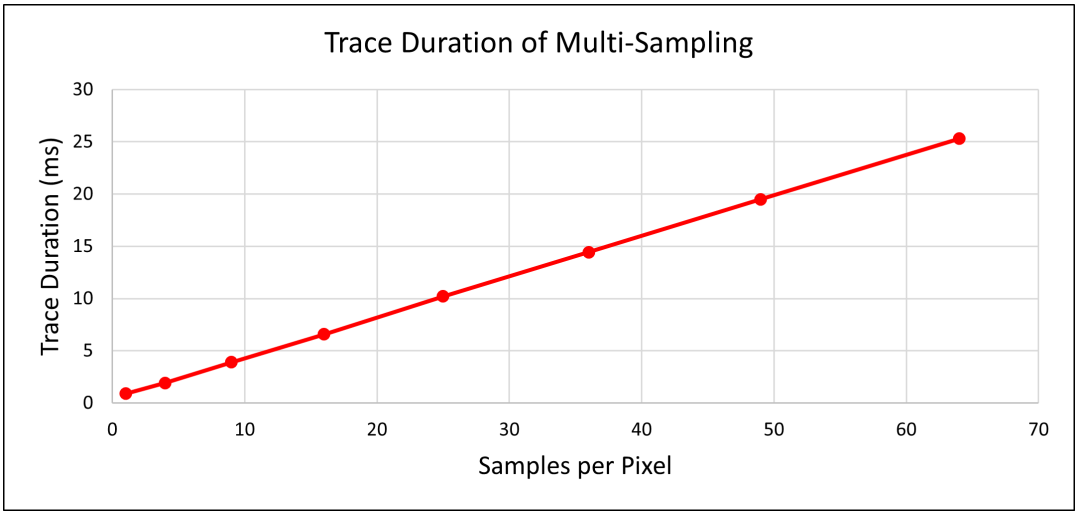


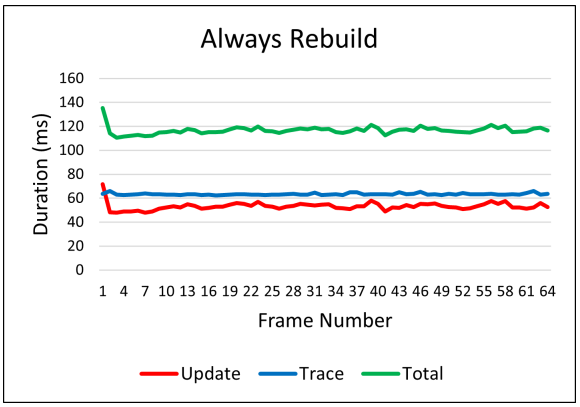
Figure 5.6: Impact of multi-sampling on ray tracing duration

This chart shows that the duration of tracing the rays scales linearly with the number of samples per pixel. This makes a lot of sence, because if we double the number of samples we also double the number of rays.

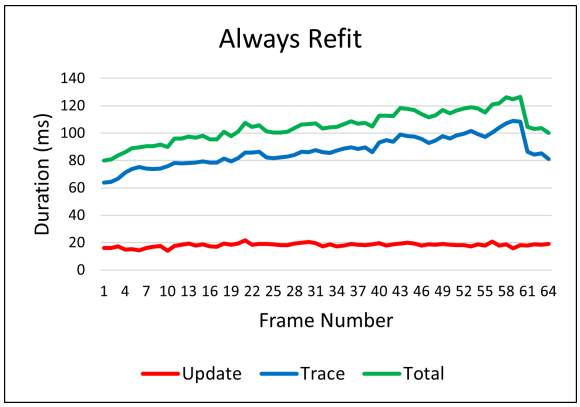
5.3 Update Strategies

We tested different strategies for updating the acceleration structure for the scene "Eternal Valley" with the standard pipeline. We generated 64 frames with a resolution of 1280×720 pixels and 64 samples per pixel and prefered a fast trace. We tested three strategies:

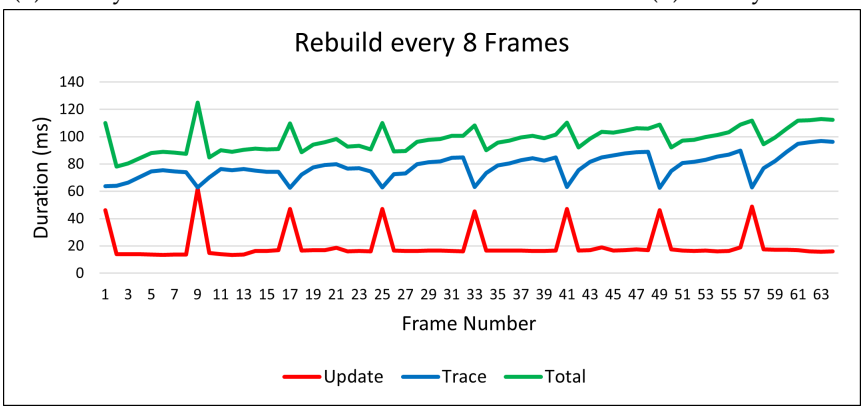
- **Always Rebuild**: The acceleration structure gets completly rebuilt every frame.
- **Always Rrefit**: After initially building the acceleration we only refit it every frame.
- **Rebuild Every Eight Frames**: We refit most frames but every eighth frame we do a rebuild.



(a) Always rebuild



(b) Always refit



(c) Rebuild every eight frames

Figure 5.7: Comparing different strategies for updating the acceleration structure

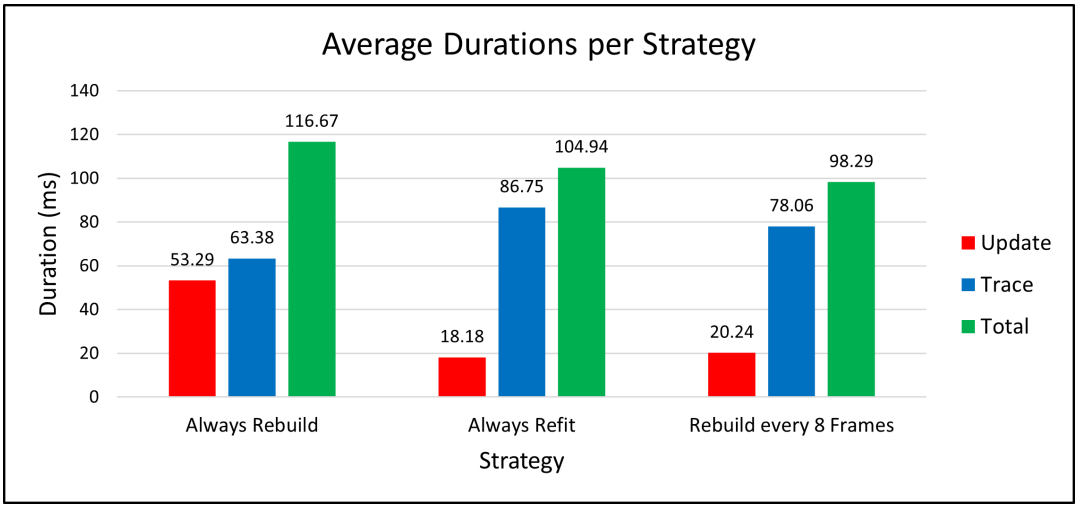


Figure 5.8: Comparing the average durations between different update strategies

5.3.1 Always Rebuild

As expected, always rebuilding gives us the shortest ray tracing duration, but it comes at the cost of a lot slower updates to the acceleration structure. The update and trace durations are very consistent and therefore the total duration also stays consistent. However, if we compare this strategy to the other strategies, it has the longest total duration on average.

5.3.2 Always Refit

This strategy results in very consistent and very short update duration. However, tracing the rays through the scenes becomes slower and slower, since the acceleration structure gets increasingly sub-optimal. This causes the stragegy to have a worse total time than always rebuilding towards the end of the animation. We can also notice, that the trace duration starts to suddenly decrease again at the end. This is probalby because the animation is looped and at the end it start to play the beginning again, for which the BVH structure is much better.

5.3.3 Rebuild Every Eight Frames

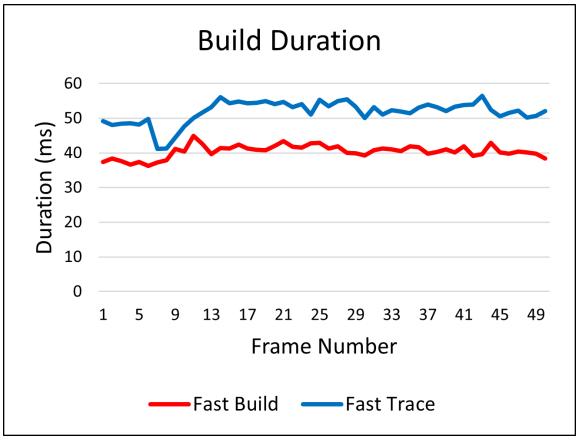
This strategy is a middle groud between the previous two. The update duration is usually very short, but every eight frames it spikes due to a rebuild occuring, which also causes the trace duration drops at those points. After each rebuild, the trace duration begins to rise again, but consistent rebuilds prevent is from diverging to far from the initial duration. This strategy has good average update and trace durations, which result in the best total duration on average compared to the other strategies.

5.3.4 Discussion

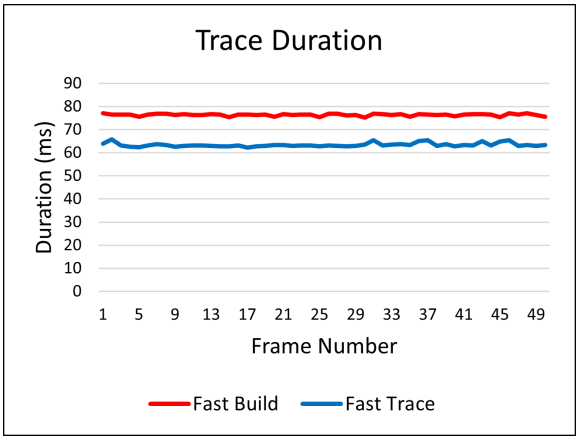
Because of the presented measurements someone might assume that the third strategy is the best one, but our measurements can not be generalized. If, for example, we keep increasing the number of rays, sooner or later the strategy of always rebuilding will become more favorable. Always refitting, however, is almost never a good strategy. Even if you are not tracing that many rays, the BVH will become increasingly sub-optimal in a lot of scenarios, which makes always refitting an unsustainable strategy. Though it might be viable for scenes, that never change to much from their initial state.

5.4 Fast Build vs. Fast Trace

We tested the flags `PREFER_FAST_BUILD` and `PREFER_FAST_TRACE` for the scene "Eternal Valley" with the standard pipeline. We generated 50 frames with a resolution of 1280×720 pixels and 64 samples per pixel. We also rebuilt the acceleration structure every frame, since these flags have no effect on refitting.



(a) Build duration



(b) Trace duration

Figure 5.9: Comparing the build and trace duration of the flag `PREFER_FAST_BUILD` to the flag `PREFER_FAST_TRACE`

.

We can see the expected outcome of `PREFER_FAST_BUILD` having shorter build durations and longer trace durations compared to `PREFER_FAST_TRACE`. It is important to mention that these flags do not impact refitting. These flags only effect the structure of the BVH, which refitting does not touch.

Whether you should use `PREFER_FAST_BUILD` or `PREFER_FAST_TRACE` depends on the specific scenario. If you are tracing a lot of rays and the trace duration is the bottleneck in your application, you should probably use `PREFER_FAST_BUILD`. On the other hand, if most of your time is spent on rebuilding (not refitting!), `PREFER_FAST_TRACE` is most likely the better choice.

6 Conclusion

This thesis aimed to understand how hardware-accelerated ray tracing can be implemented using Vulkan. We explored how acceleration structures, the ray tracing pipeline, and the shader binding table work together to enable hardware-accelerated ray tracing. We also looked at how a mesh can be represented by a bottom-level acceleration structure, and how a node in a scene graph that uses a mesh can be represented as an instance in the top-level acceleration structure. This approach means we do not have to update the entire acceleration structure; instead, we only need to update the meshes that change by rebuilding or refitting their BLAS, and we may need to update the TLAS if the transforms of some nodes have changed.

We also explored how acceleration structures can be updated either by completely rebuilding them or by only refitting the volumes of the BVH to the new positions. In our testing, a combination of both methods—where we refit most of the time and periodically rebuild—proved to be the most efficient in terms of average total rendering time. In many scenarios, this is likely the best approach. However, rendering duration can vary significantly from frame to frame, and a rebuild can cause visible spikes in rendering time. In a real-time application, we would need a way to smooth out these spikes, for example, by rebuilding different meshes across different frames, which is possible thanks to the two-level hierarchy of the acceleration structure. We also have the option to rebuild less dynamic meshes less frequently and more dynamic ones more often, which may further improve performance. This strategy may not always be ideal. If a large number of rays are cast in the application, it may be better to rebuild the acceleration structures every frame to optimize tracing performance. In these situations, setting the `PREFER_FAST_TRACE` flag often makes sense.

However, casting only a few rays relative to the number of dynamic objects does not automatically justify refitting every frame, because refitting is not sustainable in many scenes. Refitting only gives good results if the scene does not move too far away from its original state. If the scene keeps changing significantly, rendering time will steadily increase.

Finally, we also saw that not all rays have equal cost. If possible, we can use flags like `gl_RayFlagsSkipClosestHitShaderEXT` and `gl_RayFlagsTerminateOnFirstHitEXT`, which can significantly reduce ray cost. As we observed, our shadow rays were about 37% faster than regular rays.

List of Figures

List of Tables

Bibliography

[App68] A. Appel. "Some techniques for shading machine renderings of solids". In: *Proceedings of the April 30–May 2, 1968, Spring Joint Computer Conference*. AFIPS '68 (Spring). Atlantic City, New Jersey: Association for Computing Machinery, 1968, pp. 37–45. ISBN: 9781450378970. URL: https://doi.org/10.1145/1468075.1468082.

[Chr+18] P. Christensen, J. Fong, J. Shade, W. Wooten, B. Schubert, A. Kensler, S. Friedman, C. Kilpatrick, C. Ramshaw, M. Bannister, B. Rayner, J. Brouillat, and M. Liani. "An Advanced Path Tracing Architecture for Movie Rendering". In: *ACM Transactions on Graphics*. Vol. 37. 3. 2018. URL: https://docslib.org/doc/639313/an-advanced-path-tracing-architecture-for-movie-rendering.

[Cor18] N. Corporation. *Turing Architecture White Paper*. Tech. rep. NVIDIA Corporation, 2018.

[GPU] GPUOpen-LibrariesAndSDKs. *Vulkan Memory Allocator*. URL: https://github.com/GPUOpen-LibrariesAndSDKs/VulkanMemoryAllocator.git (visited on 09/17/2025).

[HVH16] C. Hery, R. Villemin, and F. Hecht. *Towards Bidirectional Path Tracing at Pixar*. Tech. rep. 2016. URL: https://graphics.pixar.com/library/BiDir/paper.pdf.

[Hue+25] R. Huerta, M. A. Shoushtary, J.-L. Cruz, and A. González. *Analyzing Modern NVIDIA GPU cores*. 2025. arXiv: 2503.20481 [cs.AR]. URL: https://arxiv.org/abs/2503.20481.

[Jac+12] A. Jacobson, I. Baran, L. Kavan, J. Popović, and O. Sorkine. "Fast Automatic Skinning Transformations". In: *ACM Transactions on Graphics (proceedings of ACM SIGGRAPH)* 31.4 (2012), 77:1–77:10.

[KA13] T. Karras and T. Aila. "Fast parallel construction of high-quality bounding volume hierarchies". In: *Proceedings of the 5th High-Performance Graphics Conference*. HPG '13. Anaheim, California: Association for Computing Machinery, 2013, pp. 89–99. ISBN: 9781450321358. DOI: 10.1145/2492045.2492055. URL: https://doi.org/10.1145/2492045.2492055.

[Kav+07] L. Kavan, S. Collins, J. Žára, and C. O'Sullivan. "Skinning with dual quaternions". In: *Proceedings of the 2007 Symposium on Interactive 3D Graphics and Games*. I3D '07. Seattle, Washington: Association for Computing Machinery, 2007, pp. 39–46. ISBN: 9781595936288. DOI: 10.1145/1230100.1230107. URL: https://doi.org/10.1145/1230100.1230107.

[KŽ05] L. Kavan and J. Žára. "Spherical blend skinning: a real-time deformation of articulated models". In: *Proceedings of the 2005 Symposium on Interactive 3D Graphics and Games*. I3D '05. Washington, District of Columbia: Association for Computing Machinery, 2005, pp. 9–16. ISBN: 1595930132. DOI: 10.1145/1053427.1053429. URL: https://doi.org/10.1145/1053427.1053429.

[Khr21] Khronos Group. *glTF™ 2.0 Specification*. The Khronos Group Inc. 2021. URL: https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html (visited on 09/22/2025).

[Khr25] Khronos Group. *Vulkan® 1.4.327 - A Specification (with all registered Vulkan extensions)*. The Khronos Group Inc. 2025. Chap. 39. Acceleration Structures. URL: https://registry.khronos.org/vulkan/specs/latest/html/vkspec.html#acceleration-structure (visited on 09/22/2025).

[Koc+20] D. Koch, T. Hector, J. Barczak, and E. Werness. *Ray Tracing in Vulkan*. 2020. URL: https://www.khronos.org/blog/ray-tracing-in-vulkan (visited on 09/25/2025).

[Kop+12] D. Kopta, T. Ize, J. Spjut, E. Brunvand, A. Davis, and A. Kensler. "Fast, effective BVH updates for animated scenes". In: *Proceedings of the ACM SIGGRAPH Symposium on Interactive 3D Graphics and Games*. I3D '12. Costa Mesa, California: Association for Computing Machinery, 2012, pp. 197–204. ISBN: 9781450311946. DOI: 10.1145/2159616.2159649. URL: https://doi.org/10.1145/2159616.2159649.

[Lau+09] C. Lauterbach, M. Garland, S. Sengupta, D. Luebke, and D. Manocha. "Fast BVH construction on gpus". In: *Computer Graphics Forum* 28 (Apr. 2009), pp. 375–384. DOI: 10.1111/j.1467-8659.2009.01377.x.

[Lun] C. Lunarg. *vk-bootstrap: Vulkan Bootstrapping Library*. URL: https://github.com/charles-lunarg/vk-bootstrap (visited on 09/19/2025).

[MT05] T. Möller and B. Trumbore. "Fast, minimum storage ray/triangle intersection". In: *ACM SIGGRAPH 2005 Courses*. SIGGRAPH '05. Los Angeles, California: Association for Computing Machinery, 2005, 7–es. ISBN: 9781450378338. DOI: 10.1145/1198555.1198746. URL: https://doi.org/10.1145/1198555.1198746.

[PL10] J. Pantaleoni and D. Luebke. "HLBVH: hierarchical LBVH construction for real-time ray tracing of dynamic geometry". In: *Proceedings of the Conference on High Performance Graphics*. HPG '10. Saarbrucken, Germany: Eurographics Association, 2010, pp. 87–95.

[Pop+06] S. Popov, J. Gunther, H.-p. Seidel, and P. Slusallek. "Experiences with Streaming Construction of SAH KD-Trees". In: *2006 IEEE Symposium on Interactive Ray Tracing*. 2006, pp. 89–94. DOI: 10.1109/RT.2006.280219.

[Sha05] Y. Shafranovich. *Common Format and MIME Type for Comma-Separated Values (CSV) Files*. RFC 4180. Oct. 2005. DOI: 10.17487/RFC4180. URL: https://www.rfc-editor.org/info/rfc4180 (visited on 09/15/2025).

[SK07] A. Soupikov and A. Kapustin. "Highly Parallel Fast KD-tree Construction for Interactive Ray Tracing of Dynamic Scenes". In: *Comput. Graph. Forum* 26 (Sept. 2007), pp. 395–404. DOI: 10.1111/j.1467-8659.2007.01062.x.

[Wal07] I. Wald. "On fast Construction of SAH-based Bounding Volume Hierarchies". In: *2007 IEEE Symposium on Interactive Ray Tracing*. 2007, pp. 33–40. DOI: 10.1109/RT.2007.4342588.

[WBS07] I. Wald, S. Boulos, and P. Shirley. "Ray tracing deformable scenes using dynamic bounding volume hierarchies". In: *ACM Trans. Graph.* 26.1 (Jan. 2007), 6–es. ISSN: 0730-0301. DOI: 10.1145/1189762.1206075. URL: https://doi.org/10.1145/1189762.1206075.

[Wal+14] I. Wald, S. Woop, C. Benthin, G. S. Johnson, and M. Ernst. "Embree: a kernel framework for efficient CPU ray tracing". In: 33.4 (July 2014). ISSN: 0730-0301. URL: https://doi.org/10.1145/2601097.2601199.

[Whi80] T. Whitted. "An improved illumination model for shaded display". In: *Commun. ACM* 23.6 (June 1980), pp. 343–349. ISSN: 0001-0782. DOI: 10.1145/358876.358882. URL: https://doi.org/10.1145/358876.358882.

[Wil+05] A. Williams, S. Barrus, R. K. Morley, and P. Shirley. "An efficient and robust ray-box intersection algorithm". In: *ACM SIGGRAPH 2005 Courses*. SIGGRAPH '05. Los Angeles, California: Association for Computing Machinery, 2005, 9–es. ISBN: 9781450378338. DOI: 10.1145/1198555.1198748. URL: https://doi.org/10.1145/1198555.1198748.

[zeu] zeux. *volk*. URL: https://github.com/zeux/volk.git (visited on 09/17/2025).